ABSTRACT

The Missing Person Identification System is an AI-based application designed to assist in the identification of missing persons by comparing facial information extracted from registered missing-person cases and public sightings.

In traditional missing-person investigation processes, authorities often depend on manual photograph comparison, paperwork and communication between different locations. These activities can become time-consuming when the number of missing-person reports and public sightings increases.

To address this problem, the proposed system provides a digital platform consisting of two separate interfaces: an Officer/Admin Portal and a Public Submission Portal.

The Officer/Admin Portal allows authorised officers to register missing-person cases by entering personal and case-related information and uploading photographs. The system uses MediaPipe Face Landmarker to detect faces and extract 468 facial landmarks. These landmarks contain three-dimensional coordinates, producing a 1,404-dimensional feature vector.

The Public Submission Portal allows members of the public to submit photographs or videos of individuals they believe may be missing. Facial information is extracted from the submitted media using the same processing technique.

The system uses the K-Nearest Neighbours (KNN) algorithm from the Scikit-learn library to compare the extracted facial feature vectors and identify potential matches.

The application also provides an administrative dashboard, geographic case visualisation using Folium, role-based access control and optional email notifications through SMTP configuration.

The system is developed using Python and Streamlit, while SQLModel and SQLite are used for data storage. OpenCV supports video processing.

The primary objective of this project is to demonstrate how Artificial Intelligence, Machine Learning and Computer Vision can support the management and review of missing-person cases.

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Background and Problem Statement
3	Aim, Objectives and Scope
4	Existing System and Proposed System
5	System Overview
6	Functional Requirements
7	Non-Functional Requirements
8	Technology Stack
9	System Architecture
10	Case Registration Module
11	Public Sighting Submission Module
12	Face Detection and Landmark Extraction
13	Face Feature Representation
14	K-Nearest Neighbours Matching
15	Case Status and Match Confirmation
16	Video Processing
17	Dashboard and Case Monitoring
18	Geographic Mapping and City Analysis
19	Email Notification Module
20	Authentication and Role-Based Access
21	Database and Data Storage
22	Privacy, Ethics and Responsible Use

23 Security Considerations
24 Installation and Environment Setup
25 Configuration and Demo Data
26 Testing Strategy
27 Test Cases and Expected Results
28 Limitations
29 Future Enhancements
30 Deployment and Maintenance
31 Results and Expected Outcomes
32 Conclusion
33 References and Documentation
34 Appendix A: Commands and Configuration
35 Appendix B: Glossary and Submission Notes

INTRODUCTION

1.1 Introduction

In today's digital world, Artificial Intelligence (AI) and Computer Vision are playing an increasingly important role in solving real-world problems. These technologies allow computers to analyse images, recognise patterns and process visual information with minimal human intervention.

One important social problem that requires efficient technological support is the issue of missing persons. Every year, many individuals, including children and adults, become separated from their families or go missing due to different circumstances.

Finding a missing person can be a difficult and time-consuming process. Families usually report missing-person cases to the police or relevant authorities. Officers collect information, photographs and other identifying details. When someone later reports seeing a person who may be missing, investigators must compare the new information with existing records.

Traditional methods of comparing photographs manually can become inefficient when there are hundreds or thousands of records.

The Missing Person Identification System is developed to address this challenge by using Artificial Intelligence, Computer Vision and Machine Learning techniques.

The system provides a digital platform where authorised officers can register missing-person cases and members of the public can submit sightings. The application extracts facial landmarks from uploaded photographs and videos and compares them using a Machine Learning algorithm.

The main purpose of the system is to help identify possible matches more efficiently and support the investigation process.

1.2 Overview of the Project

The Missing Person Identification System is a Python-based application that provides two separate portals:

1. Officer/Admin Portal.
2. Public Submission Portal.

The Officer/Admin Portal is designed for authorised users. Officers can register missing-person cases, upload photographs and view their registered records. Administrators have additional permissions to manage cases and initiate the matching process.

The Public Submission Portal allows members of the public to submit photographs or videos of individuals they believe may be missing. This portal does not require a login.

The application uses MediaPipe Face Landmarker to extract facial information. It generates a 468-point facial mesh containing three-dimensional coordinates.

The extracted information is converted into a numerical feature vector containing 1,404 values.

The K-Nearest Neighbours algorithm compares these feature vectors to identify possible similarities between registered cases and public sightings.

The system also provides a dashboard with case statistics, geographic mapping and optional email notifications.

1.3 Need for the Project

The need for this project arises from the limitations of traditional missing-person investigation methods.

Some major challenges include:

- Manual comparison of photographs.
- Difficulty managing a large number of missing-person records.
- Delays in identifying possible sightings.
- Lack of centralised digital information.
- Difficulty monitoring cases across different cities.
- Time-consuming communication with complainants.

1.4 Importance of Artificial Intelligence

Artificial Intelligence allows computer systems to perform tasks that generally require human intelligence.

In this project, AI-related computer vision techniques are used to detect faces and extract facial landmarks.

Machine Learning is used to compare the numerical representations of faces and identify possible similarities.

This demonstrates how AI technologies can be applied to a socially meaningful problem.

1.5 Purpose of the System

The main purpose of the system is to create a digital platform that supports missing-person case management and public sighting comparison.

The system aims to:

- Simplify missing-person case registration.
- Allow public users to submit sightings.
- Extract facial landmarks automatically.
- Compare facial feature vectors using KNN.
- Display possible matches.
- Monitor case information through a dashboard.
- Provide optional email notifications.

1.6 Organisation of the Report

This report is organised into 35 chapters covering the introduction, problem statement, objectives, system requirements, architecture, implementation modules, technologies, database, testing, limitations and future enhancements.

It also contains configuration instructions, references and technical appendices.

BACKGROUND AND PROBLEM STATEMENT

2.1 Background

Missing-person cases are a serious social concern. When a person goes missing, family members and authorities need to collect information and search for possible locations where the individual may have been seen.

Photographs are among the most important pieces of information used during this process.

However, comparing photographs manually becomes difficult when the number of missing-person reports and public sightings increases.

In many situations, a person may be seen by someone who does not know that the individual has been reported missing. If the sighting is not communicated to the appropriate authority, an important lead may be missed.

Digital technologies can help organise this information and make comparison more efficient.

The Missing Person Identification System uses facial landmark extraction and Machine Learning to support this process.

2.2 Existing Challenges

The following are some of the major challenges associated with traditional missing-person management.

1. Manual photograph comparison

Investigators may need to compare multiple photographs manually. This process can take considerable time.

2. Information management

Missing-person details may be collected from different sources. Managing these records without a centralised platform can create difficulties.

3. Delayed identification

When a public sighting is reported, identifying the corresponding missing-person record may take time.

4. Geographic tracking

Understanding the distribution of unresolved cases across different cities can be difficult without proper visualisation.

2.3 Problem Statement

The main problem addressed by this project is the difficulty of efficiently comparing missing-person photographs with photographs or videos submitted by members of the public.

Traditional manual comparison methods require significant human effort and become increasingly difficult when the number of records grows.

Therefore, a digital system is required that can:

- Store missing-person case information.
- Accept public sighting submissions.
- Automatically detect faces.
- Extract facial landmark information.
- Compare facial feature vectors.
- Identify potential matches.
- Display case statistics and geographic information.

The proposed system uses Computer Vision and Machine Learning techniques to support these activities.

2.4 Proposed Solution

The proposed solution is a web-based application developed using Python and Streamlit.

It provides separate interfaces for officers, administrators and public users.

When an officer registers a missing-person case, the system processes the uploaded photograph and extracts facial landmarks.

Similarly, when a public user submits a sighting, the system extracts facial landmarks from the submitted image or video.

The extracted feature vectors are stored locally in the database.

An administrator can initiate the matching process using the KNN algorithm. The algorithm compares the available facial vectors and identifies possible similarities.

The application also provides case status information and optional email notifications.

2.5 Expected Benefits

The proposed system provides several expected benefits:

1. Reduces repeated manual photograph comparison.
2. Provides a structured digital case management workflow.
3. Allows public participation through a separate submission portal.
4. Supports automated facial feature extraction.
5. Provides similarity-based matching.
6. Helps administrators monitor case statistics.
7. Provides geographic visualisation of cases.

2.6 Social Importance

This project demonstrates how technology can support public welfare.

By providing a platform for organised case registration and public sighting submissions, the system illustrates a possible way to improve information management and assist authorised investigations.

However, the system is an assistance tool. A similarity result must be verified by authorised personnel before any identification is treated as confirmed.

AIM, OBJECTIVES AND SCOPE

3.1 Aim of the Project

The main aim of the Missing Person Identification System is to develop an AI-based application that assists in identifying potential missing-person matches by comparing facial landmark information extracted from registered cases and public sightings.

The system combines Computer Vision, Machine Learning and database technologies to provide an organised digital workflow.

3.2 Primary Objectives

The primary objectives of the project are:

1. To develop a digital platform for registering missing-person cases.
2. To provide a separate public portal for submitting sightings.
3. To implement automatic face detection using MediaPipe.
4. To extract 468 facial landmarks from detected faces.
5. To convert facial landmarks into numerical feature vectors.
6. To implement K-Nearest Neighbours for similarity comparison.
7. To display possible matching results.
8. To provide an administrative dashboard.
9. To display case locations using interactive maps.
10. To support optional email notifications.

3.3 Technical Objectives

The technical objectives include:

- Developing the application using Python.
- Creating user interfaces using Streamlit.
- Implementing facial landmark extraction using MediaPipe Tasks.
- Using Scikit-learn for KNN-based comparison.
- Managing records using SQLModel.
- Storing data in SQLite.

- Processing videos using OpenCV.
- Displaying geographic information using Folium.
- Implementing password hashing using bcrypt.

3.4 Scope of the Project

The project focuses on missing-person case management and potential face-based matching.

The scope includes the following modules.

Officer Module

Allows authorised officers to register missing-person cases and view their own records.

Administrator Module

Allows administrators to manage cases, view all records, initiate matching and edit or delete cases.

Public Submission Module

Allows members of the public to submit photographs or videos of possible sightings without logging in.

Face Detection Module

Detects faces and extracts 468 facial landmarks.

Matching Module

Uses KNN to compare stored facial feature vectors.

Dashboard Module

Displays case counts and geographic summaries.

Notification Module

Provides optional email notifications through SMTP configuration.

3.5 Users of the System

The system has three main user categories.

User	Main Responsibilities
Administrator	Manage cases, initiate matching, edit and delete records
Officer	Register cases and view own cases
Public User	Submit possible missing-person sightings

3.6 Limitations of Scope

The project is primarily an educational demonstration.

It does not establish that the facial matching algorithm can independently identify individuals with legal or investigative certainty.

The supplied documentation does not specify a production deployment model, benchmark accuracy or complete legal compliance framework.

The system should therefore be tested and reviewed before being used with real personal information.

EXISTING SYSTEM AND PROPOSED SYSTEM

4.1 Existing System

In traditional missing-person management, information is collected through reports, photographs and communication between families and authorities.

Officers may manually examine photographs and compare them with available records.

When a member of the public reports a possible sighting, the information may need to be checked against existing missing-person cases.

This process can become difficult when a large number of records are involved.

4.2 Disadvantages of the Existing System

The major disadvantages include:

1. Manual photograph comparison takes time.
2. Managing large numbers of records is difficult.
3. Information may be distributed across different locations.
4. Tracking case status requires additional effort.
5. Geographic case analysis may be difficult.
6. Public sighting information may not be compared immediately.

4.3 Proposed System

The proposed Missing Person Identification System provides a digital platform that combines case registration, public submissions, facial feature extraction and similarity matching.

The system uses two independent Streamlit applications that share the same SQLite database.

The first application is designed for officers and administrators.

The second application is designed for public users.

When an officer registers a missing-person case, the system extracts facial landmarks from the uploaded photograph.

When a public user submits a sighting, the same face-processing method is applied.

The KNN algorithm compares the extracted vectors to identify possible matches.

4.4 Advantages of the Proposed System

- Automated facial landmark extraction.
- Digital case registration.
- Separate public submission interface.
- Image and video submission support.
- KNN-based facial similarity comparison.
- Interactive geographic mapping.
- Role-based access control.
- Optional email notifications.
- Local database storage.

4.5 Comparison Between Existing and Proposed Systems

Existing System	Proposed System
Manual photograph comparison	Automated feature extraction and similarity comparison
Paper-based or separate records	Digital database
Manual sighting collection	Public submission portal
Difficult case monitoring	Dashboard-based monitoring
Limited geographic visualisation	Interactive map
Manual communication	Optional email notifications
Time-consuming comparison	KNN-based candidate matching

4.6 Conclusion

The proposed system provides a more organised approach to missing-person case management.

By combining facial landmark extraction, similarity matching and digital record management, the application aims to support authorised users in reviewing possible missing-person matches.

SYSTEM OVERVIEW

5.1 Introduction

The Missing Person Identification System is developed using Python and several open-source technologies.

The application consists of two main portals that perform different responsibilities while sharing a common database.

The Officer/Admin Portal handles missing-person registration and case management.

The Public Submission Portal accepts sightings from members of the public.

Both portals use the same facial feature extraction process to ensure that registered cases and public sightings can be compared.

5.2 Main Components

The major components of the system are:

1. Officer/Admin Portal

The portal provides authenticated access to authorised users.

It allows officers to register cases and administrators to manage the overall system.

2. Public Submission Portal

This portal allows members of the public to submit possible sightings without requiring authentication.

3. Face Detection Component

MediaPipe Face Landmarker detects faces and extracts facial landmarks.

4. Feature Extraction Component

The extracted landmarks are converted into numerical vectors containing 1,404 values.

5. Matching Component

Scikit-learn KNN compares the extracted vectors and identifies possible similarities.

6. Database Component

SQLModel and SQLite manage the stored records.

7. Mapping Component

Folium provides interactive geographic visualisation.

8. Notification Component

SMTP supports optional email notifications.

5.3 Working Process

The general working process is as follows:

1. An officer logs into the application.
2. The officer registers a missing-person case.
3. The system accepts the photograph.
4. MediaPipe extracts facial landmarks.
5. The resulting feature vector is stored.
6. A public user submits a possible sighting.
7. The system processes the submitted image or video.
8. The extracted feature vector is stored.
9. An administrator initiates matching.
10. KNN compares the available vectors.
11. Potential matches are displayed.
12. A confirmed match can trigger a status update and email notification.

5.4 System Workflow Diagram

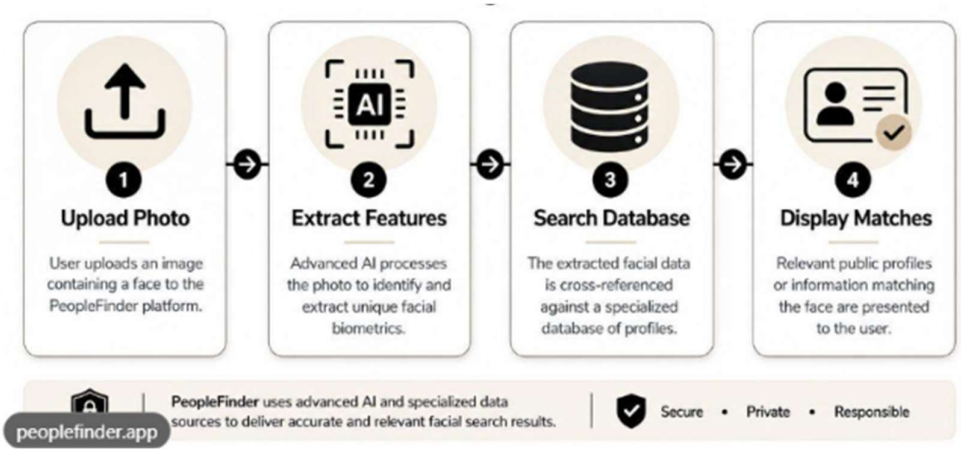


Figure 5.1: General workflow of the Missing Person Identification System.

5.5 System Operation

The application is started through two separate Streamlit commands.

Officer/Admin Portal:

streamlit run Home.py

Public Submission Portal:

streamlit run mobile_app.py

Both applications use the same SQLite database file.

5.6 Data Storage

The application uses SQLite for local data storage.

The database file is named:

sqlite_database.db

The application also stores image files inside the resources directory.

The README states that the database and image storage are excluded from version control and that the project processes face-derived data locally.

README(2).md

5.7 Offline Functionality

The face landmarker model is downloaded automatically during the first use. The model is approximately 30 MB.

After the model has been downloaded and cached, both portals can operate offline according to the project documentation.

FUNCTIONAL REQUIREMENTS

6.1 Introduction

Functional requirements describe the operations and services that a software application must provide to its users.

The Missing Person Identification System contains several functional modules that work together to provide missing-person registration, public sighting submission, facial feature extraction and matching.

6.2 User Authentication

The system provides a login mechanism for authorised officers and administrators.

Users must enter their configured username and password to access the internal portal.

Passwords are stored as bcrypt hashes in the login configuration file.

The authentication module identifies the user's role and provides access according to the assigned permissions.

6.3 Missing-Person Case Registration

The system allows authorised officers to register missing-person cases.

The registration process includes:

- Entering case-related information.
- Providing personal details.
- Entering location information.
- Uploading a photograph.
- Extracting facial landmarks.
- Saving the case information.

The uploaded photograph is processed automatically using MediaPipe.

6.4 Public Sighting Submission

The public portal allows visitors to submit information about possible missing-person sightings.

The portal does not require login authentication.

Users can upload photographs or videos containing a person who may have been reported missing.

The application processes the submitted media and stores the extracted information for later comparison.

6.5 Facial Landmark Extraction

The application uses MediaPipe Face Landmarker to detect faces and extract 468 facial landmarks.

Each landmark contains three-dimensional coordinate information.

The extracted information is converted into a numerical feature vector.

6.6 Face Matching

The system uses the K-Nearest Neighbours algorithm to compare facial feature vectors.

The administrator can initiate the matching process through the Match Cases functionality.

The application identifies potential similarities between registered cases and public sightings.

6.7 Case Management

Administrators can:

- View all registered cases.
- Edit case information.
- Delete records.
- Initiate matching.
- Monitor case status.

Officers can register cases and view their own records.

NON-FUNCTIONAL REQUIREMENTS

7.1 Introduction

Non-functional requirements describe the quality, performance, security and reliability expected from a software application.

These requirements help ensure that the Missing Person Identification System operates effectively and provides a usable experience.

7.2 Performance

The application should process uploaded photographs and videos within a reasonable amount of time.

Face detection and feature extraction should be performed efficiently.

The KNN matching process should be capable of comparing available records without unnecessary delays.

The actual performance depends on image quality, hardware configuration and dataset size.

7.3 Reliability

The application should handle invalid files and processing errors without unexpectedly terminating.

Database operations should preserve registered case and sighting information.

The application should provide understandable error messages when a face cannot be detected.

7.4 Usability

The system should provide a simple and understandable user interface.

Officers should be able to register cases without requiring advanced technical knowledge.

The public portal should be easy to access from mobile devices.

7.5 Security

The application should protect sensitive information through authentication and role-based access.

Passwords should be stored using secure hashing.

The login configuration file should not be uploaded to public repositories.

TECHNOLOGY STACK

8.1 Introduction

The Missing Person Identification System uses several open-source technologies.

Each technology performs a specific responsibility in the application. The combination of these tools allows the system to provide a digital interface, process facial information, compare feature vectors, store records and display geographic information.

8.2 Python

8.3 Streamlit

8.4 MediaPipe Tasks

8.5 Scikit-learn

8.6 SQLModel

8.7 SQLite

8.8 Folium

8.9 OpenCV

8.10 Bcrypt

8.11 Technology Summary

Technology	Purpose
Python	Main programming language
Streamlit	Web application interface
MediaPipe	Face landmark extraction
Scikit-learn	KNN matching
SQLModel	Database modelling
SQLite	Data storage
Folium	Geographic mapping
OpenCV	Video processing
Bcrypt	Password hashing

SYSTEM ARCHITECTURE

9.1 Introduction

System architecture describes the overall structure of an application and explains how its different components communicate with each other.

The Missing Person Identification System follows a modular architecture in which separate components handle user interaction, face processing, matching, database operations and notifications.

The application consists of two independent Streamlit portals that share a common database.

9.2 Architectural Components

The system architecture contains the following major layers.

1. Presentation Layer

The presentation layer provides the user interface.

It contains:

- Officer/Admin Portal.
- Public Submission Portal.

The Officer/Admin Portal provides authenticated access to internal users.

The Public Submission Portal allows members of the public to submit possible sightings.

2. Application Layer

The application layer handles the main workflow.

It receives user input, validates information and coordinates communication between different components.

3. Face Processing Layer

This layer uses MediaPipe Face Landmarker to detect faces and extract facial landmarks.

OpenCV supports video-frame extraction.

CASE REGISTRATION MODULE

10.1 Introduction

The Case Registration Module is one of the most important components of the Missing Person Identification System.

It allows authorised officers to register missing-person cases and upload photographs for facial landmark extraction.

The module ensures that case information is stored digitally and can be used during the matching process.

10.2 Purpose of the Module

The main purpose of this module is to create a structured record for every reported missing-person case.

It collects relevant information and associates the uploaded photograph with the corresponding case.

The extracted facial information is used later for comparison with public sightings.

10.3 Working Process

The case registration process follows these steps:

1. The officer opens the Officer/Admin Portal.
2. The officer enters valid login credentials.
3. The system verifies the user.
4. The officer opens the registration page.
5. The required case details are entered.
6. A photograph is uploaded.
7. The system detects the face.
8. MediaPipe extracts facial landmarks.
9. The application generates a feature vector.
10. The case information is saved in the database.

PUBLIC SIGHTING SUBMISSION MODULE

11.1 Introduction

The Public Sighting Submission Module is designed to allow members of the public to report possible sightings of missing persons.

In many situations, individuals may recognise someone who appears in a missing-person notice. However, they may not know how to communicate this information to the appropriate authorities.

This module provides a simple digital platform where people can submit photographs or videos of possible sightings.

11.2 Purpose of the Module

The main purpose of this module is to collect public sighting information and make it available for comparison with registered missing-person cases.

Unlike the Officer/Admin Portal, the public submission portal does not require users to log in.

This makes the submission process easier for people who want to report a possible sighting.

11.3 Features of the Module

The main features include:

1. Public access without login.
2. Photograph upload.
3. Video upload.
4. Automatic face detection.
5. Facial landmark extraction.
6. Storage of sighting information.
7. Availability of submissions for matching.

FACE DETECTION AND LANDMARK EXTRACTION

12.1 Introduction

Face detection and landmark extraction are important parts of the Missing Person Identification System.

The application uses MediaPipe Face Landmarker to detect faces in uploaded photographs and video frames.

After detecting a face, the system extracts facial landmarks that represent different geometric positions on the face.

These landmarks are used to generate numerical information for the matching algorithm.

12.2 What is Face Detection?

Face detection is a Computer Vision technique used to identify the presence and location of human faces in images or video frames.

It helps a computer determine whether a face exists in visual data.

In this project, face detection is performed using the MediaPipe Face Landmarker model.

12.3 What are Facial Landmarks?

Facial landmarks are specific geometric points located across different regions of a face.

These points can represent areas such as:

- Eyes.
- Eyebrows.
- Nose.
- Mouth.
- Face outline.
- Cheeks.

MediaPipe Face Landmarker extracts 468 facial landmarks in the project.

12.4 MediaPipe Face Landmarker

MediaPipe is an open-source framework developed by Google.

FACE FEATURE REPRESENTATION

13.1 Introduction

After extracting facial landmarks, the system converts the information into a numerical representation called a feature vector.

Machine Learning algorithms generally require structured numerical data as input.

Therefore, the facial landmarks extracted by MediaPipe are converted into a numerical format before being used for comparison.

13.2 Feature Vector

A feature vector is an ordered collection of numerical values representing selected characteristics of an object.

In this project, the feature vector is generated from the three-dimensional coordinates of the facial landmarks.

The system uses 468 landmarks, with each landmark containing three coordinates.

13.3 Feature Vector Calculation

The total number of values is calculated as follows:

$$ N = L \times C $$

Where:

- N = Total numerical values.
- L = Number of facial landmarks.
- C = Coordinates per landmark.

Substituting the values:

$$ N = 468 \times 3 $$

$$ N = 1404 $$

Therefore, each face is represented using a 1,404-dimensional vector.

13.4 Example of Feature Representation

A simplified example of facial landmark data is shown below:

face_landmarks = [[0.45, 0.32, 0.01], [0.47, 0.34, 0.02], [0.49, 0.36, 0.01],]

K-NEAREST NEIGHBOURS MATCHING

14.1 Introduction

The K-Nearest Neighbours algorithm is a Machine Learning technique used to identify nearby data points based on their feature values.

The Missing Person Identification System uses KNN to compare facial feature vectors extracted from missing-person photographs and public sightings.

The main objective is to identify potential similarities between the two datasets.

14.2 What is KNN?

K-Nearest Neighbours is an instance-based Machine Learning algorithm.

It identifies the nearest data points to a given input based on a distance measurement.

The value of K represents the number of neighbouring data points considered during the comparison.

For example, if K is set to 3, the algorithm considers the three nearest data points.

14.3 Role of KNN in the Project

In this project, KNN compares the 1,404-dimensional facial feature vectors.

The system maintains facial information extracted from registered missing-person cases and public sightings.

When an administrator initiates the matching process, the algorithm compares the available vectors and identifies possible similarities.

14.4 Working Process

The matching process follows these steps:

1. Load registered case feature vectors.
2. Load public sighting feature vectors.
3. Prepare the numerical feature data.
4. Initialise the KNN model.
5. Compare the available feature vectors.
6. Identify nearby vectors.
7. Generate matching results.

CASE STATUS AND MATCH CONFIRMATION

15.1 Introduction

The Case Status and Match Confirmation Module manages the status of missing-person records after the matching process.

It helps administrators monitor whether a registered case remains unresolved or has been marked as found.

15.2 Purpose

The main purpose of this module is to provide a clear view of case progress and associate possible sightings with registered cases.

The application provides case information and status details through the case management interface.

15.3 Matching Workflow

When an administrator clicks Match Cases, the application compares available public sightings with open missing-person records.

The KNN algorithm processes the extracted facial feature vectors.

If the system identifies a sufficiently close candidate, it follows the documented status-update process.

15.4 Case Status

The project documentation describes the following status categories:

Status	Description
Not Found	The case has not been marked as resolved
Found	The case has been marked as found

The dashboard displays counts of found and not-found cases.

15.5 Match Information

The case management interface can display information associated with a potential match.

The README describes the view_cases screenshot as showing:

- Found status.
- Sighting location.

DASHBOARD AND CASE MONITORING

17.1 Introduction

The Dashboard and Case Monitoring Module provides an overview of missing-person case information.

It allows administrators and authorised users to understand the current status of registered cases.

The dashboard presents important information through counts, visual elements and geographic summaries.

17.2 Purpose of the Dashboard

The primary purpose of the dashboard is to make case information easier to understand.

Instead of opening individual records to understand the overall situation, users can view important case statistics in one place.

17.3 Dashboard Features

According to the project documentation, the dashboard includes:

1. Total found cases.
2. Total not-found cases.
3. Geographic case distribution.
4. Map visualisation across India.
5. Case monitoring information.

17.4 Case Statistics

The dashboard displays the number of cases in different status categories.

For example:

Case Category	Description
Found Cases	Cases marked as found
Not-Found Cases	Cases that remain unresolved

These statistics help administrators understand the current case situation.

SECURITY CONSIDERATIONS

23.1 Introduction

Security is an important requirement for any application that stores personal information.

The Missing Person Identification System includes user authentication, password hashing and role-based access control.

These features help protect the internal application from unauthorised access.

23.2 Authentication Security

The system uses configured credentials to authenticate officers and administrators.

Only authorised users should be allowed to access the internal portal.

23.3 Password Hashing

Bcrypt is used to generate password hashes.

This prevents the application from needing to store user passwords as plain text.

23.4 Configuration File Protection

The login configuration is stored in:

login_config.yml

The README states that this file is git-ignored by default.

Real usernames, passwords and other sensitive credentials should not be published in a public repository.

23.5 Database Security

The database contains case and sighting information.

The database file should be protected against unauthorised access.

Appropriate file-system permissions should be applied to the database and image directories.

23.6 Public Upload Security

The public portal allows unauthenticated users to submit images and videos.

This introduces additional security considerations.

The application should validate:

INSTALLATION AND ENVIRONMENT SETUP

24.1 Introduction

The Missing Person Identification System is developed using Python and several supporting libraries.

The application requires a compatible Python environment and the dependencies specified in the project's requirements.txt file.

The README provides a straightforward installation and execution procedure.

24.2 System Requirements

The following are general requirements for running the application.

Component	Requirement
Operating System	Windows, Linux or macOS
Programming Language	Python
Interface Framework	Streamlit
Database	SQLite
Face Processing	MediaPipe
Matching	Scikit-learn
Video Processing	OpenCV
Internet	Required for initial model download

The exact Python version should be checked against the project's dependency configuration.

24.3 Step 1: Obtain the Project

Download or clone the project repository.

Open the project directory in a terminal or command prompt.

cd Finding-missing-person-using-AI

24.4 Step 2: Install Dependencies

Install the required Python libraries using:

pip install -r requirements.txt

This command installs the dependencies listed in requirements.txt.

24.5 Step 3: Run the Officer/Admin Portal

Execute the following command:

streamlit run Home.py

The Streamlit application starts the Officer/Admin Portal.

Users can access the interface through the local address provided by Streamlit.

24.6 Step 4: Run the Public Submission Portal

Open another terminal window and execute:

streamlit run mobile_app.py

This starts the Public Submission Portal.

24.7 Step 5: Database Creation

The SQLite database is created automatically when the application is first used, according to the project README.

The database file is:

sqlite_database.db

24.8 Step 6: Face Landmarker Model Download

The face landmarker model is approximately 30 MB.

It is downloaded automatically during the first use and cached locally.

After the model is downloaded, the application can operate offline according to the README.

24.9 Step 7: Optional Email Configuration

To enable email notifications, configure the following environment variables:

SMTP_HOST=

SMTP_PORT=

SMTP_USER=

SMTP_PASSWORD=

Provide the appropriate SMTP details for the email service.

CONFIGURATION AND DEMO DATA

25.1 Introduction

The project includes configuration and demonstration-data utilities.

These utilities allow developers to configure login credentials and populate the database with demonstration records.

They are useful during development, testing and project demonstrations.

25.2 Login Configuration

User credentials are maintained in:

login_config.yml

The file stores the username, display name, email address, city, area, role and bcrypt password hash.

25.3 Generating Password Hashes

The README demonstrates generating a bcrypt password hash using Python.

import bcryptpassword = b"your_password"hashed = bcrypt.hashpw(password, bcrypt.gensalt())print(hashed.decode())

The generated hash can be added to the login configuration file.

25.4 Demo Image Folders

The project provides two folders for demonstration images:

scripts/bulk_data/reported/

This folder contains images representing missing-person cases.

scripts/bulk_data/publicly_seen/

This folder contains images representing public sightings.

25.5 Downloading Sample Images

The project provides a utility named:

scripts/download_celebrity_images.py

The README describes using DuckDuckGo image search to download demonstration images.

CONCLUSION

32.1 Conclusion

The Missing Person Identification System is a Python-based application developed to demonstrate the use of Artificial Intelligence, Computer Vision and Machine Learning in addressing the problem of missing-person identification.

The project provides a digital platform for registering missing-person cases and collecting public sightings.

It consists of two separate interfaces: an Officer/Admin Portal and a Public Submission Portal.

The Officer/Admin Portal allows authorised officers to register missing-person cases, upload photographs and manage case information. Administrators can view records and initiate the matching process.

The Public Submission Portal allows members of the public to submit photographs or videos of possible sightings without requiring authentication.

The system uses MediaPipe Face Landmarker to extract 468 facial landmarks from detected faces. Each landmark contains three-dimensional coordinate information, producing a total of 1,404 numerical values.

These values are used to generate facial feature vectors.

The Scikit-learn K-Nearest Neighbours algorithm compares these vectors and identifies potential similarities between registered missing-person cases and public sightings.

The application also uses SQLModel and SQLite for data storage, OpenCV for video processing and Folium for geographic visualisation.

Optional email notifications are provided through SMTP configuration.

The project demonstrates how multiple open-source technologies can be integrated into a single application to solve a socially meaningful problem.

It also provides valuable practical experience in Python programming, Computer Vision, Machine Learning, database management and web application development.

However, automated facial similarity matching has limitations. Image quality can affect the results, and a similarity score does not establish identity with certainty. Therefore, human verification and appropriate privacy and security safeguards remain essential.

REFERENCES AND DOCUMENTATION

The following resources are relevant to the development and technical understanding of the Missing Person Identification System.

33.1 Project Documentation

1. Missing Person Identification System, supplied project README file.
2. Project installation and configuration instructions.
3. Project technology stack and feature documentation.
4. Project workflow and demonstration documentation.

33.2 Programming Language Documentation

Python Official Documentation

Website: https://docs.python.org/3/

Used for understanding Python programming concepts, libraries and application development.

33.3 Framework Documentation

Streamlit Documentation

Website: https://docs.streamlit.io/

Used for developing interactive web applications using Python.

33.4 Computer Vision Documentation

MediaPipe Documentation

Website: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

Used for understanding facial landmark detection and extraction.

OpenCV Documentation

Website: https://docs.opencv.org/

Used for understanding image and video processing.

33.5 Machine Learning Documentation

Scikit-learn Documentation

Website: https://scikit-learn.org/stable/